Blueprint — UX Strategy from First Principles

Stated assumptions (no primary research exists yet — everything below is hypothesis, to be validated with real users): the primary user is a developer or tech lead facing a codebase they don't fully understand — onboarding onto it, reviewing it, assessing its risk, or deciding where to change it. Their goal is not "see data about my repo." Their goal is *comprehension they can act on*.

The core reframe

The brief says "entering the mind of an AI software architect." Accept the metaphor, but discipline it, because it has a trap on each side:

- **The trap of theater.** If "entering the mind" means performed intelligence — pulsing brain visuals, fake thinking animations, an AI that emotes — that is Simulated Understanding (Category 8, Medium) and drifts toward Anthropomorphic Manipulation (High). Spectacle creates awe, and awe *distances* the user from the material. Rejected.
- **The trap of the dashboard.** If "repository intelligence" means grids of metric cards — commit counts, language pie charts, contributor graphs — that is inventory, not intelligence. A dashboard answers "what is there?" The user's actual question is "what does it *mean*, and what should I do?" Rejected, per the brief and per principle.

The disciplined version of the metaphor: **you are not watching an AI perform thought — you are walking through the model of the codebase that the AI has already built, with the architect beside you.** The product is the architect's *understanding*, made navigable. Every claim in it can be opened to reveal its reasoning and traced to the code that supports it. "Entering the mind" means *transparency of reasoning*, not *theater of cognition*.

Everything below derives from that.

---

1. Primary user emotion

Target: earned lucidity — the "oh, I see it now" moment. The fog-lifting feeling of a complex system suddenly having a graspable shape. Secondary: **calibrated trust** — confidence in the architect's read because its reasoning is inspectable, and honest doubt where the architect itself is uncertain.

Explicitly rejected emotions:

- **Awe / "wow."** Awe is a launch-video emotion. It decays in a week and it positions the user as spectator. Lucidity positions them as the thinker.

(Principle: this also guards against Manufactured Dependency — the product should grow the user's competence in their own codebase, not replace it.)

- **Anxiety / urgency.** A risk-detection product can easily become a fear machine (red badges, alarm counts). Fear drives engagement but corrodes judgment. Findings are presented as an architect presents them: consequential, sourced, calm.

Test for every screen: *does the user leave knowing something about their codebase they didn't know before, and could they defend that knowledge to a colleague?* If they can only say "the tool said so," we produced dependency, not lucidity.

2. Product personality

A senior architect who has already read the whole codebase — calm, precise, opinionated with evidence, and honest about uncertainty.

Concretely:

- **Has a thesis.** It doesn't present neutral data and make you interpret; it says "this service is the load-bearing wall; these three modules are where change is risky, because…" Opinion is the product. Undefended opinion is not — every thesis carries its evidence.
- **Shows its work, always.** Every judgment is expandable: claim → reasoning → the actual code/commits it rests on. This is the anti-pattern defense built into the personality: no Undisclosed AI Decisions (Category 8, High).
- **Calibrated, not confident.** "I'm certain," "this is likely," and "I couldn't determine this" are three different statements and the product must distinguish them structurally, not in fine print. An architect who never says "I don't know" is a salesman.
- **Not a character.** No name-as-persona, no emotions, no "I missed you." First person is acceptable only as the voice of reasoning ("I traced this dependency…"), never of feeling.

3. Interaction philosophy

Dialogue over inspection; interrogation over consumption.

- **Question-first.** The primary act in Blueprint is asking — not clicking through a feature taxonomy. "What breaks if I change the auth module?" "Why is this service structured this way?" Every artifact the system shows is framed as an *answer*, and answers invite follow-ups.
- **Every claim is a handle.** Nothing the architect asserts is terminal. Any statement can be pulled: *why do you believe this?* → *show me the evidence* → *take me to the code.* Three tugs from any thesis to raw source, always.

This is progressive disclosure of reasoning — sequence, never hide (Principle 3).

- **Investigations are durable objects.** A line of questioning is not a chat that scrolls into oblivion; it's a *thread* — resumable, shareable, revisitable. The user who gets interrupted for 20 minutes (real conditions, Principle 2) returns to the exact state of their inquiry.
- **The system never acts silently.** Re-analysis, model updates, new findings — all announced with what changed and why, never mutated under the user's feet. Transparent consequences (Principle 1).

4. Information hierarchy

Interpretation above evidence above inventory — ordered by consequence, never by data type.

1. **The thesis** — the architect's current read of this repository: what it is, how it's shaped, where it's healthy, where it's fragile. One screen, articulable in one sentence.
2. **What changed / what's notable** — deltas in the architect's understanding since the user last looked, ranked by consequence to *this user's* work, not by recency or count.
3. **Evidence** — the reasoning layer, revealed on demand under any claim.
4. **Inventory** — files, metrics, raw graphs. Present, reachable, never the lead. Metrics appear *in service of a claim* ("this module churns 4× more than its neighbors — that's why I flag it") or not at all.

Confidence is first-class metadata at every layer: a low-confidence finding never sits visually equal to a verified one.

The dashboard test, made precise: **if a screen presents numbers whose interpretation is left to the user, it has failed the hierarchy.** That's the Real Estate Tour anti-pattern (Category 9) applied to data — inventory masquerading as intelligence.

5. Mental model

Rejected model: *"a monitoring dashboard for my repo"* (glance, scan widgets, leave).

Proposed model: "a living model of my codebase, built by an architect who studied it — and I walk through that model with them." Like touring a building with the engineer who did the structural survey: you move through it, ask about anything, and get answers grounded in inspection.

The nouns this implies — and this is the deepest break from the current product:

- The atomic units are **not files and folders**. They are *systems, boundaries, decisions, risks, and questions*. Files are evidence, not architecture.
- A repo in Blueprint is a **study** (an evolving body of understanding), not a "project page."
- A session is an **investigation**, not a monitoring visit. It has a question, a path, and a conclusion the user takes away.
- Time exists: the architect's understanding **evolves**, and that evolution is itself content ("since last month, this boundary has eroded").

6. Workspace architecture

Reject the grid-of-cards. The architecture is **one canvas, one orientation layer, one evidence rail** — focus plus context, never twelve competing panels.

Three spaces per study, each answering one question:

- **The Briefing** — *"What does the architect think?"* The current thesis and what's changed. This is the arrival point. It replaces "overview/dashboard" and is prose-and-claims, not widgets. Purpose in one sentence: catch me up on my codebase's state as a considered read, not a readout.
- **The Atlas** — *"What is the shape of this system?"* The navigable structural model: systems, boundaries, dependencies, at controllable altitude (system → module → file → line). Spatial, explorable, and every region openable into "what the architect knows about this."
- **The Threads** — *"What am I trying to find out?"* The user's investigations: active, resumed, concluded. This is where dialogue lives and persists.

The **evidence rail** is cross-cutting: from any claim, in any space, it opens to show reasoning and source without losing your place. Global level: a plain list of studies — whichever study you were last thinking in, you resume mid-thought, not at a portfolio dashboard.

7. Navigation philosophy

Navigate by intent and altitude, not by feature taxonomy.

- **Asking is the universal entry.** From anywhere, the user can pose a question and be taken to the answer — the primary nav is a verb, not a menu. The three spaces exist as stable ground, but the expected path between them is a question, not a tab-click.
- **Altitude replaces pages.** Movement through the codebase model is zoom — system down to line — rather than hopping between unrelated screens. The user always knows their current altitude and location: wayfinding is non-

negotiable (Principle 3), because a "mind" you get lost inside is a failure, not an aesthetic.

- **Back retraces reasoning.** History is the path of the investigation ("thesis → this claim → this evidence → this file"), and the user can walk back up it. How-I-got-here is always answerable.
- **No more than three persistent destinations.** Briefing, Atlas, Threads. Any proposed fourth section must prove it isn't one of these wearing a new label — accretion of nav items is how dashboards grow back.

What must be validated before this hardens

Per Principle 4, these are hypotheses: (1) that users want a *thesis* rather than self-serve data — test with 5 target users comparing a briefing-first vs. metrics-first prototype; (2) that question-first navigation beats browsable structure for first-contact users — tree-test the Atlas against task prompts; (3) the trust calibration — do users correctly distinguish the architect's certain vs. uncertain claims? If they can't, the confidence layer is decoration and must be redesigned.

Natural next steps when you're ready: /journey to design the first-contact flow (connecting a repo → first Briefing — the moment the entire emotional bet is won or lost), and /articulate to define the architect's voice, since in this strategy the *words are the product*.